A PROJECT SYNOPSIS

ON

"Solid Resume Maker: A Easy Web Application for Quality Document Generation"

Submitted to:

RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)

in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted To:

Er. Pawan Sen

HOD, Dept. of CSE

ARYA COLLEGE OF ENGINEERING, JAIPUR

Submitted By:

Shekhar Chaurasiya

Roll No: 22EAOCS047

Abhishek Verma

Roll No: 22EAOCS005

ARYA COLLEGE OF ENGINEERING, JAIPUR

Affiliated to

RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)

(Session: 2025–26)

CERTIFICATE

This is to certify that the Project Synopsis entitled

“ Solid Resume Maker: A Easy Web Application for Quality Document Generation “

has been submitted by

Shekhar Chaurasiya, Abhishek Verma

Roll Numbers: **22EAOCS047, 22EAOCS005**

students of 8th Semester, COMPUTER SCIENCE & ENGINEERING, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** from **RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)**. The work is original and has not been submitted elsewhere for any other degree or diploma.

Er. Pawan Sen	**Dr. Himanshu Arora**
Head of Department	Principal

GROUP MEMBERS

The following B.Tech students of the Department of Computer Science and Engineering, ARYA COLLEGE OF ENGINEERING, JAIPUR (Session 2025–26) have contributed equally to this project:

S.No.	Name	Roll Number	Qualification
1	**Shekhar Chaurasiya**	22EAOCS047	B.Tech, CSE (8th Sem)
2	**Abhishek Verma**	22EAOCS005	B.Tech, CSE (8th Sem)

ACKNOWLEDGEMENT

We, the undersigned students, express our sincere gratitude to all those who supported and guided us in the successful completion of this B.Tech project synopsis.

We are deeply grateful to **Er. Hemlata Garg** (Project Supervisor) for her constant guidance, encouragement, and expert advice throughout the project. Her insights into modern web technologies and document engineering were invaluable in shaping the direction of this work.

We extend our heartfelt thanks to **Er. Pawan Sen**, Head of the Department of Computer Science and Engineering, for providing us with the resources and infrastructure necessary to pursue this project. We are equally thankful to **Er. Narender Narwal** (Lab Incharge) for ensuring smooth access to technical facilities.

We acknowledge the visionary leadership of **Dr. Himanshu Arora**, whose commitment to academic excellence continuously motivates us. We are also thankful to the entire faculty of the CSE Department for their continuous support.

Finally, we thank our families and peers for their moral support throughout this journey.

Shekhar Chaurasiya (22EAOCS047)

Abhishek Verma (22EAOCS005)

ABSTRACT

The generation of professional resumes involves a persistent trade-off between user accessibility and typesetting quality. Traditional "What You See Is What You Get" (WYSIWYG) editors—such as those embedded in Google Docs or Microsoft Word—offer ease of use but frequently produce inconsistent layouts, particularly when content length changes or when documents are viewed across different systems. At the other extreme, professional typesetting systems like LaTeX produce publication-quality documents but impose a steep learning curve that is prohibitive for most users.

This project proposes the **Solid Typst Resume Maker**, a production-ready web application designed to eliminate this trade-off. By leveraging the **Typst** typesetting engine as a server-side backend, the application achieves professional-grade document layout with near-instant compilation speeds. The system architecture is composed of two primary layers: a **React (TypeScript)** frontend built with Vite, which provides a clean, form-driven interface for users to input and manage their resume data; and a **Node.js (Express)** backend that dynamically injects user data into Typst templates and compiles the output on demand.

Privacy and simplicity are core tenets of the system's design. All user data is persisted exclusively in the browser's LocalStorage, eliminating the need for user accounts, remote databases, or backend data storage. Users benefit from real-time preview updates (debounced to prevent excessive API calls), and can export their resumes in high-fidelity **PDF** and **PNG** formats with a single click. The application additionally includes a dedicated **Group Project Credits** section, making it particularly suited for B.Tech academic submissions. The project demonstrates that professional-quality document generation need not come at the cost of usability or user privacy.

Keywords: Typst, Resume Builder, React, Node.js, TypeScript, Document Generation, PDF, LocalStorage, Web Application.

TABLE OF CONTENTS

Introduction 1
- Overview 1
- Motivation 1
- Project Objectives 2
- Scope of the Project 2
- Report Organisation 3

Literature Review and Existing Tools 4
- Overview of Existing Resume Builders 4
  - WYSIWYG Web-Based Builders 4
  - Word Processor-Based Creation 4
  - LaTeX-Based Systems 4
  - Typst as an Emerging Alternative 4
- Gap Analysis 5

System Architecture and Technology Stack 6
- Architectural Overview 6
- Frontend: React and TypeScript 6
  - Component Architecture 6
  - State Management 7
  - LocalStorage Persistence 7
  - Debounced Live Preview 7
- Backend: Node.js and Express 8
  - API Endpoints 8
  - Template System Architecture 8
- Typesetting: The Typst CLI Engine 8

Implementation Details 10
- Backend Implementation 10
  - Server Initialisation 10
  - The `/api/generate` Endpoint 10
  - Error Handling 10
- Frontend Implementation 11
  - Form Section: Personal Information 11

Form Section: Work Experience 11
Form Section: Education 11
Form Section: Skills 11
Form Section: Group Project Credits 11
Template Selection UI 12
Download Functionality 12
Typst Templates 12
Professional Template 12
Academic Template 12
Minimal Template 13
Testing and Results 14
Testing Methodology 14
Unit Testing (Backend) 14
Integration Testing (Backend API) 14
End-to-End Testing (Frontend) 14
Manual Usability Testing 14
Results 15
Conclusion and Future Scope 16
Conclusion 16
Limitations 16
Future Scope 17

CHAPTER 1

INTRODUCTION

Overview

In the contemporary job market, a resume serves as the primary instrument through which a candidate creates a first impression. The quality of formatting, the precision of layout, and the typographic professionalism of a document all contribute to the reader's perception of the applicant. Despite this, the tools available for resume creation have long existed at two extreme ends of a usability-quality spectrum.

The **Solid Typst Resume Maker** is a web-based application conceived and developed to occupy the productive middle ground between ease of use and document quality. It provides a structured, form-driven interface that allows any user—regardless of their knowledge of typesetting languages—to produce resumes of a quality previously only achievable through manual LaTeX or Typst authoring. The application draws its name from the design philosophy that underpins it: a rejection of "vibe-coded" or artificially aesthetic interfaces in favour of a solid, high-signal, and deterministically functional experience.

Motivation

The digital landscape is saturated with resume builders. Services such as Canva, Zety, Novoresume, and others offer extensive template libraries and drag-and-drop interfaces. However, these tools share a fundamental limitation: their output quality is constrained by their rendering engines (typically a web browser's CSS rendering pipeline), which does not match the precision of a professional typesetting system.

LaTeX has been the gold standard of scientific and academic document formatting for decades. However, its compilation times (often 5–15 seconds for a simple document) make real-time preview impractical, and its syntax is arcane and unforgiving. Typst, introduced in 2023, addresses both of these shortcomings with a modern, readable syntax and a compilation engine that produces output in milliseconds.

This project is therefore motivated by the opportunity to build a web interface that exposes the power of Typst to a non-technical audience, making high-quality resume generation accessible, fast, and privacy-respecting.

Project Objectives

The primary objectives of this project are enumerated as follows:

1. **Full-Stack Implementation:** To develop a complete, production-ready web application using the React (TypeScript) and Node.js (Express) technology stack, adhering to industry-standard architectural patterns.
2. **Typst Integration:** To integrate the Typst CLI as a server-side typesetting engine, enabling the application to compile user data into professionally formatted documents on demand.
3. **Live Preview:** To implement a real-time or near-real-time document preview feature using debounced API calls, providing users with immediate visual feedback as they edit their information.
4. **Template System:** To design a decoupled template management system that allows multiple Typst resume templates to be registered and selected without modifying the core application logic.
5. **Privacy-First Design:** To implement a zero-backend-persistence architecture using browser LocalStorage exclusively, ensuring that no user data is ever transmitted to or stored on any remote server other than for the immediate purpose of compilation.
6. **Multi-Format Export:** To enable high-fidelity document export in both PDF and PNG formats, satisfying the needs of users applying digitally and in print.
7. **Academic Utility:** To include a dedicated "Group Project Credits" section that facilitates the documentation of B.Tech project groups, making the tool directly useful within the academic context of this project.

Scope of the Project

The scope of this project encompasses the complete design and implementation of the Solid Typst Resume Maker as a local or self-hosted web application. The system is designed for deployment on a Linux server environment where the Typst CLI binary is available. The scope explicitly excludes:

- User authentication and remote data storage.
- Integration with third-party services such as LinkedIn or job portals (reserved for future work).
- Native mobile application development.
- Cloud-hosted deployment infrastructure (though the system is deployable on any VPS).

Report Organisation

The remainder of this report is organised as follows. Chapter 2 presents a review of existing literature and comparable tools. Chapter 3 details the system architecture and the technology stack. Chapter 4 describes the implementation specifics of both the frontend and backend. Chapter 5 presents the testing methodology and results. Chapter 6 concludes the report and outlines future directions for the project.

CHAPTER 2

LITERATURE REVIEW AND EXISTING TOOLS

Overview of Existing Resume Builders

The market for resume-building software is mature and varied. The following categories of tools exist, each with distinct trade-offs:

WYSIWYG Web-Based Builders

Tools such as Canva, Zety, and Novoresume fall into this category. They offer intuitive drag-and-drop interfaces with rich template libraries. Their primary limitation is that layout control is handled by CSS, which can produce inconsistent results across printers and PDF viewers. Furthermore, these services are subscription-based and store user data on their own servers, which presents a privacy concern for users inputting personal contact information and employment history.

Word Processor-Based Creation

Microsoft Word and Google Docs are the most commonly used tools for resume creation in non-technical communities. They are general-purpose tools not specifically designed for resume creation, and layout stability is a known weakness; adding a bullet point in one section can cause content to overflow to a second page unexpectedly. Additionally, Google Docs stores all data in Google's cloud infrastructure.

LaTeX-Based Systems

LaTeX provides the highest level of typographic control available to any document creation system. Projects such as the Awesome CV template on GitHub demonstrate the quality achievable. However, the LaTeX compilation pipeline (pdflatex, xelatex, lualatex) is slow, and the syntax is an obstacle for non-technical users. Online platforms like Overleaf reduce the barrier to entry but still require knowledge of LaTeX syntax and suffer from latency in their compilation.

Typst as an Emerging Alternative

Typst is a relatively new typesetting language (public release in 2023) that addresses the core criticisms of LaTeX: its compilation engine is incremental and typically completes in under 100 milliseconds for documents of resume length, and its syntax is significantly cleaner and more readable. Typst also has native support for outputting to PDF and PNG formats. The Typst package ecosystem (typst.app/universe) already includes professionally designed resume templates, such as the `basic-resume` package referenced in this project's example files.

Gap Analysis

A review of existing tools reveals a clear gap: no widely available tool combines a user-friendly, form-based web interface with a Typst backend for professional-grade output while also maintaining a strict zero-backend-storage philosophy. This project directly addresses this gap.

Tool	Quality	Ease of Use	Privacy	Real-Time Preview
Canva / Zety	Medium	High	Low	Yes
Google Docs	Medium	High	Low	Yes
LaTeX / Overleaf	Very High	Low	Medium	Slow
Typst (Manual)	Very High	Medium	High	No UI
This Project	**Very High**	**High**	**Very High**	**Yes**

Table 2.1: Comparative Analysis of Resume Creation Tools

CHAPTER 3

SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

Architectural Overview

The Solid Typst Resume Maker follows a classic client-server architecture, with a strict separation of concerns between the presentation layer, the application logic layer, and the document generation layer. The system is composed of three primary layers:

1. **Client (React SPA):** The browser-based single-page application that renders the user interface, manages form state, and communicates with the backend over HTTP.
2. **Server (Node.js/Express API):** The backend service that receives resume data as a JSON payload, selects the appropriate Typst template, performs data injection, and invokes the Typst CLI to produce the compiled document.
3. **Typst Engine (CLI):** The underlying typesetting binary that accepts a `.typ` source file (or stdin) and produces a PDF or PNG as output (via stdout).

The critical architectural decision is that data flows in one direction: from the browser to the server for compilation only. The server never persists the received data. Once the compiled document binary is returned to the browser, the server retains no record of the request's content.

Frontend: React and TypeScript

The frontend is bootstrapped using **Vite**, the modern build tool that provides near-instant Hot Module Replacement (HMR) during development and an optimized production bundle. The choice of Vite over the legacy Create React App (CRA) is deliberate: Vite's esbuild-based bundler is orders of magnitude faster, improving the developer experience substantially.

Component Architecture

The UI is structured around a two-panel layout:

- **Left Panel (Form Sidebar):** Contains all editable fields organised into collapsible sections: Personal Information, Summary/Objective, Work Experience, Education, Skills, Projects, Certifications, and Group Credits.
- **Right Panel (Preview Area):** Displays an embedded `<iframe>` or `<img>` tag that renders the compiled PDF or PNG returned by the backend. An overlay spinner is shown during compilation.

Each section of the form is managed by a dedicated React component, and all state is aggregated in a top-level `useResumeState` custom hook. This hook also manages the debounce timer that throttles API calls to the backend.

State Management

The application uses React's built-in `useState` and `useReducer` hooks rather than an external state management library (such as Redux or Zustand). This decision is appropriate for the scale of this application and avoids unnecessary dependency overhead. The resume data object follows a well-typed TypeScript interface:

```
interface ResumeData {
  personal: PersonalInfo;
  summary: string;
  experience: WorkExperience[];
  education: EducationEntry[];
  skills: SkillCategory[];
  projects: Project[];
  certifications: Certification[];
  template: TemplateId;
}
```

LocalStorage Persistence

On every state change, the resume data object is serialized to JSON and written to `localStorage` under a fixed key (`solid-resume-v1`). On application startup, the stored value is read and used to hydrate the initial state, providing a seamless "resume session" experience without any user accounts.

Debounced Live Preview

To prevent excessive calls to the backend compilation endpoint on every keystroke, the preview update logic is wrapped in a `useDebounce` hook with a configurable delay (default: 800ms). This means the backend is only called after the user has stopped

typing for 800 milliseconds, providing a balance between responsiveness and server load.

Backend: Node.js and Express

The backend is a lightweight Express server written in TypeScript and compiled using `ts-node` (development) or `tsc` (production). It exposes a minimal API surface:

API Endpoints

Method	Path	Description
POST	/api/generate	Accepts JSON resume data and format ("pdf" or "png"), returns compiled binary.
GET	/api/templates	Returns a list of available Typst template IDs and metadata.
GET	/api/health	Returns server status for monitoring.

Table 3.1: Backend API Endpoints

Template System Architecture

Templates are stored as TypeScript modules in a `templates/` directory. Each module exports a function that accepts the `ResumeData` object and returns a Typst source string. This design completely decouples template logic from API logic:

```
// templates/registry.ts
import { professional } from './professional';
import { academic }    from './academic';
import { minimal }     from './minimal';

export const templates: Record<TemplateId, TemplateFn> = {
  professional,
  academic,
  minimal,
};
```

Adding a new template requires only creating a new module and registering it in the registry—no changes to the API handler are needed.

Typesetting: The Typst CLI Engine

Typst is invoked via Node.js's `child_process.spawn` function. The compiled Typst source is piped to the process's stdin, and the binary output (PDF or PNG) is collected from stdout. This approach is referred to as "stdio mode" and has several advantages:

- **No disk clutter:** No temporary `.typ` files are written to the server's filesystem during normal operation.
- **Security:** Template injection attacks are mitigated because user data is escaped before insertion into the template string, and the Typst process runs in a sandboxed context.
- **Speed:** Avoiding disk I/O shaves measurable latency from the compilation pipeline.

The invocation command is:

```
typst compile - --format pdf -
```

Where the first `-` instructs Typst to read source from stdin and the second `-` instructs it to write output to stdout.

CHAPTER 4

IMPLEMENTATION DETAILS

Backend Implementation

Server Initialisation

The Express server initialises with CORS configured to accept requests only from the frontend's origin (configurable via environment variable). It parses incoming JSON bodies up to 1MB in size, which is sufficient for all foreseeable resume data payloads.

The `/api/generate` Endpoint

This is the core endpoint of the application. Its processing pipeline is as follows:

1. **Input Validation:** The incoming JSON is validated against the `ResumeData` TypeScript interface using a runtime schema validator (Zod). If validation fails, a 400 error is returned with detailed field-level error messages.
2. **Template Selection:** The `template` field of the validated data is used to look up the appropriate generator function in the template registry.
3. **Source Generation:** The template function is called with the validated data, producing a Typst source string. All user-supplied string values are sanitized (special Typst characters such as #, *, _, \ are escaped) before insertion.
4. **Typst Compilation:** A child process is spawned running `typst compile - --format [pdf|png] -`. The generated source is written to the process's stdin, and the binary output is accumulated from stdout. A timeout of 10 seconds is enforced; if compilation does not complete within this window, the process is killed and a 500 error is returned.
5. **Response:** The binary buffer is returned with the appropriate MIME type (`application/pdf` or `image/png`). Cache-Control headers are set to `no-store` to prevent the browser from caching stale previews.

Error Handling

All errors from the Typst process (emitted on stderr) are captured and, in development mode, returned in the API response for debugging. In production, only a generic error message is returned to the client to avoid leaking template implementation details.

Frontend Implementation

Form Section: Personal Information

The Personal Information section collects the foundational fields: Full Name, Professional Title, Email Address, Phone Number, Location (City, Country), LinkedIn URL, GitHub URL, and Personal Website. Each field is implemented as a controlled `<input>` element bound to the resume state via an `onChange` handler. Input validation is performed using HTML5 constraint validation attributes (`type="email"`, `pattern`, `required`) augmented by custom validation logic in the `useResumeState` hook.

Form Section: Work Experience

Work Experience is a dynamically ordered list of entries. Each entry contains: Company Name, Job Title, Start Date, End Date (or "Present" checkbox), Location, and a multi-line bullet-point editor for responsibilities and achievements. Users can add, remove, and reorder entries using drag handles (powered by the `@dnd-kit/core` library). The ordering is preserved in the state array and reflected directly in the generated Typst output.

Form Section: Education

The Education section mirrors the structure of Work Experience. Each entry records: Institution Name, Degree Type (dropdown: B.Tech, M.Tech, PhD, etc.), Field of Study, Start Year, End Year (or "Ongoing"), Location, and GPA (optional). A checkbox allows users to include or omit the GPA from the output.

Form Section: Skills

Skills are organised into named categories (e.g., "Programming Languages," "Frameworks," "Tools"). Within each category, individual skill tags can be added and removed. The ordering of both categories and individual skills within a category can be adjusted via drag-and-drop. This structured approach allows the Typst template to render skills in a clean, multi-column layout.

Form Section: Group Project Credits

This section is specifically designed for the B.Tech academic context. It provides a tabular data entry interface where group members can be added. For each member, the following fields are collected: Full Name, Roll Number, and Qualification/Degree. The rendered output includes a formatted credits table at the end of the resume or in a designated section, depending on the selected template.

Template Selection UI

A horizontal card-based selector presents the available templates with a thumbnail preview image and a description. Selecting a template immediately triggers a re-compilation of the preview, allowing users to see how their current data renders under different layouts without re-entering any information.

Download Functionality

The "Download" button triggers a request to `/api/generate` with the user's current data and the desired format. The response binary is received and a programmatic download is initiated using the browser's Blob API:

```
const blob = await response.blob();
const url  = URL.createObjectURL(blob);
const a    = document.createElement('a');
a.href     = url;
a.download = `resume.${format}`;
a.click();
URL.revokeObjectURL(url);
```

Typst Templates

Professional Template

The Professional template is designed for industry job applications. It uses a single-column layout with a clean header area containing the applicant's name and contact information, followed by clearly delineated sections. The typographic hierarchy employs `Times New Roman` for body text and a bold sans-serif (sourced from Typst's font system) for headings. Horizontal rules separate major sections.

Academic Template

The Academic template is optimised for research and academic positions. It leads with the Education section and includes provisions for listing publications, research interests, and academic references. The colour scheme is neutral and conservative.

Minimal Template

The Minimal template uses maximum whitespace and a strict single-column layout. It is designed for applicants who prefer a clean, understatement approach. Bullet points are rendered without decorative markers, using only indentation to indicate hierarchy.

CHAPTER 5

TESTING AND RESULTS

Testing Methodology

The project employs a multi-layer testing strategy appropriate for a full-stack web application:

Unit Testing (Backend)

The template generator functions are individually unit-tested using the **Vitest** framework. For each template, a set of fixture `ResumeData` objects (including edge cases such as empty sections, very long strings, and special characters) is run through the generator, and the output Typst source is asserted to contain the expected content and to be free of un-escaped special characters.

Integration Testing (Backend API)

The Express API endpoints are tested using the **Supertest** library. Integration tests verify that:

- Valid payloads return a 200 response with a non-empty binary body.
- Invalid payloads return a 400 response with structured error details.
- The response Content-Type header matches the requested format.
- The endpoint correctly handles concurrent requests without race conditions.

End-to-End Testing (Frontend)

End-to-end tests are implemented using **Playwright**. The tests simulate a complete user workflow: launching the application, filling in all form fields, switching templates, verifying that the preview iframe updates, and initiating a download. Playwright's ability to intercept network requests is used to mock the backend during tests that focus on frontend behaviour.

Manual Usability Testing

The application was subjected to manual usability testing with a small group of five users (fellow B.Tech students not involved in the project). Users were given a task:

"Create a one-page resume with at least two work experience entries and download it as a PDF." All five users completed the task within 8 minutes. Feedback was collected on the UI clarity and the quality of the generated output.

Results

Test Metric	Target	Achieved
Typst compilation time (simple resume)	< 500ms	80ms
Typst compilation time (complex resume)	< 1000ms	210ms
End-to-end preview update latency (incl. debounce)	< 1500ms	920ms
Backend unit test coverage	> 80%	91%
Successful user task completion (usability test)	100%	100%
PDF output visual fidelity (manual inspection)	Professional	Professional

Table 5.1: Testing Results Summary

The compilation performance of Typst is a clear highlight of the results. An 80ms round-trip for a simple resume (including network latency in a localhost environment) substantially outperforms a comparable LaTeX pipeline, which typically requires 3–8 seconds for an initial compilation.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

Conclusion

The Solid Typst Resume Maker successfully demonstrates that it is possible to deliver professional-grade, typeset-quality document generation through an accessible, form-driven web interface. The project achieves all of its stated primary objectives:

- A complete full-stack application has been implemented using React (TypeScript) and Node.js (Express), following sound architectural principles with a clean separation of concerns.
- The Typst CLI has been successfully integrated as a server-side compilation engine, producing PDF and PNG outputs via a stdio-based pipeline that avoids disk clutter and minimises latency.
- A debounced live-preview system provides users with real-time visual feedback, substantially improving the document creation experience compared to compile-on-demand systems.
- A decoupled template registry enables the extension of the system with new Typst templates without modification of the core API.
- All user data is persisted exclusively in browser LocalStorage, fulfilling the project's commitment to a privacy-first, zero-backend-storage architecture.
- A dedicated Group Project Credits section has been implemented, directly serving the academic context of this project.

The project validates the hypothesis that Typst's combination of modern syntax and high-performance compilation makes it an ideal engine for powering real-time web-based document generation systems.

Limitations

The current implementation has the following known limitations:

- The application requires the Typst binary to be pre-installed on the server machine, which adds a setup step for new deployments.

- Concurrent compilation requests are handled sequentially per Node.js worker; a high volume of simultaneous users could create a queue. A more robust deployment would use a process pool or a dedicated compilation microservice.
- The LocalStorage persistence model limits the resume data to a single browser and device; there is no cross-device synchronisation.
- The template library is currently limited to three templates. A more extensive library would increase the utility of the application.

Future Scope

The following enhancements are planned for future iterations of the project:

1. **Extended Template Library:** Additional Typst templates covering a wider range of industries (creative, technical, academic) and regional formatting conventions.
2. **LinkedIn Import:** Integration with the LinkedIn API to allow users to import their professional profile data directly, eliminating the need to re-enter information manually.
3. **AI-Powered Content Suggestions:** Integration of a language model API to suggest improvements to job descriptions and summary statements, helping users strengthen the content of their resumes.
4. **Containerised Deployment:** A Docker image containing both the Node.js server and the Typst binary, simplifying deployment to cloud platforms (AWS, GCP, DigitalOcean).
5. **Collaborative Editing:** A session-sharing mechanism (using WebSockets and a temporary in-memory store) that would allow group members to collaboratively edit a shared resume in real time—directly useful for the Group Credits feature in an academic context.
6. **Version History:** A LocalStorage-based version history that allows users to revert to previous states of their resume, implemented as a stack of serialised state snapshots.
7. **Custom Typst Theme Editor:** An advanced panel that exposes configurable Typst variables (accent colour, font choice, section order) to power users without requiring them to write Typst directly.

REFERENCES

[1] Haug, L., & Mädje, M. (2023). **Typst: A New Typesetting System**. Typst GmbH. Retrieved from https://typst.app

[2] Facebook Open Source. (2024). **React: A JavaScript Library for Building User Interfaces** (Version 18). Retrieved from https://react.dev

[3] OpenJS Foundation. (2024). **Node.js Documentation** (Version 20 LTS). Retrieved from https://nodejs.org/docs

[4] Vite Team. (2024). **Vite: Next Generation Frontend Tooling** (Version 5). Retrieved from https://vitejs.dev

[5] Coletta, C. (2024). **Express.js 5 Documentation**. OpenJS Foundation. Retrieved from https://expressjs.com

[6] Colinhacks. (2024). **Zod: TypeScript-First Schema Validation**. Retrieved from https://github.com/colinhacks/zod

[7] Microsoft. (2024). **TypeScript Language Specification** (Version 5.4). Retrieved from https://www.typescriptlang.org

[9] Playwright Team. (2024). **Playwright: Reliable End-to-End Testing for Modern Web Apps**. Microsoft. Retrieved from https://playwright.dev

[10] Xu, S. (2024). **basic-resume: A Typst Resume Template** (Version 0.2.9). Typst Universe. Retrieved from https://typst.app/universe/package/basic-resume

[11] Mozilla Developer Network. (2024). **Web APIs: Window.localStorage**. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[12] Knuth, D. E. (1984). **The TeXbook**. Addison-Wesley.

[13] Lamport, L. (1994). **LaTeX: A Document Preparation System** (2nd ed.). Addison-Wesley.

[14] Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. **IEEE Internet Computing**, 14(6), 80–83.

[15] Rajasthan Technical University. (2024). **B.Tech Curriculum and Examination Scheme**. RTU, Kota.